

Experiment No. 1**Experiment Name - Implementation of Fundamental Image Processing Operations using Python and OpenCV**

✓
 Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

✓
 1. Install and import the required libraries, including OpenCV, NumPy, and Matplotlib.

```
# Installing the required libraries
!pip install opencv-python numpy matplotlib
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (5.0.0.93)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
# Importing the required libraries
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
print("OpenCV Version:", cv2.__version__)
```

```
Libraries imported successfully!
OpenCV Version: 4.14.0
```

✓
 2. Load a color image from the local system and display it using both OpenCV and Matplotlib.

```
from google.colab import files
uploaded = files.upload()
```

my photo.jpg
my photo.jpg(image/jpeg) - 12084 bytes, last modified: 8/20/2026 - 100% done
 Saving my photo.jpg to my photo.jpg

```
# Loading the image using OpenCV
image = cv2.imread("my photo.jpg")

# Checking whether the image was loaded successfully or not
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully!")
```

```
Image loaded successfully!
```

```
from google.colab.patches import cv2_imshow

# Displaying image using OpenCV
display_image = cv2.resize(image, (300, 225))
print("Image displayed using OpenCV:")
cv2_imshow(image)
```

Image displayed using OpenCV:



```
# OpenCV reads images in BGR format.  
# Matplotlib expects images in RGB format.  
# Convert BGR to RGB because Matplotlib uses RGB  
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
print("Image displayed using Matplotlib:")
```

```
plt.figure(figsize=(3, 2))  
plt.imshow(image_rgb)  
plt.title("Original Color Image - Matplotlib")  
plt.axis("off")  
plt.show()
```

Image displayed using Matplotlib:

Original Color Image - Matplotlib



✓ **3. Examine the image properties such as dimensions, resolution, number of channels, and data type.**

```
# Finding image dimensions and properties  
  
height, width, channels = image.shape  
  
print("Image Properties:\n")  
print("Width:", width, "pixels")  
print("Height:", height, "pixels")  
print("Number of Channels:", channels)  
print("Image Shape:", image.shape)  
print("Data Type:", image.dtype)  
print("Total Number of Pixels:", height * width)
```

Image Properties:

```
Width: 217 pixels  
Height: 300 pixels  
Number of Channels: 3  
Image Shape: (300, 217, 3)  
Data Type: uint8  
Total Number of Pixels: 65100
```

✓ **4. Save the image in different file formats (e.g., JPEG and PNG) and compare the outputs.**

```
# Save the image in JPEG and PNG formats  
  
jpeg_path = "output_image.jpg"  
png_path = "output_image.png"  
  
# Save image  
cv2.imwrite(jpeg_path, image)  
cv2.imwrite(png_path, image)  
  
print("Image saved successfully in JPEG and PNG formats.")
```

Image saved successfully in JPEG and PNG formats.

```
# Checking file sizes
import os

jpeg_size = os.path.getsize(jpeg_path)
png_size = os.path.getsize(png_path)

print("Images saved successfully.")
print()
print("JPEG file size:", jpeg_size, "bytes")
print("PNG file size :", png_size, "bytes")
```

Images saved successfully.

JPEG file size: 15895 bytes
PNG file size : 73172 bytes

```
# Read the saved images

jpeg_image = cv2.imread("output_image.jpg")
png_image = cv2.imread("output_image.png")

# Displaying both images
plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(jpeg_image, cv2.COLOR_BGR2RGB))
plt.title("JPEG Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(png_image, cv2.COLOR_BGR2RGB))
plt.title("PNG Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

JPEG Image

PNG Image



5. Convert the image from RGB/BGR to Grayscale, HSV, and LAB color spaces.

```
# Convert the BGR image into grayscale

gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Original image shape:", image.shape)
print("Grayscale image shape:", gray_image.shape)

plt.figure(figsize=(3, 2))

plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image:\n")
plt.axis("off")

plt.show()
```

Original image shape: (300, 217, 3)
Grayscale image shape: (300, 217)

Grayscale Image:



```
# Convert BGR image to HSV color space

hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

print("HSV image shape:", hsv_image.shape)

plt.figure(figsize=(3, 2))

plt.imshow(cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB))
plt.title("HSV Image:\n")
plt.axis("off")

plt.show()
```

HSV image shape: (300, 217, 3)

HSV Image:



```
# Convert BGR image to LAB color space

lab_image = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

print("LAB image shape:", lab_image.shape)

plt.figure(figsize=(3, 2))

plt.imshow(cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB))
plt.title("LAB Color Space:\n")
plt.axis("off")

plt.show()
```

LAB image shape: (300, 217, 3)

LAB Color Space:



```
# Displaying all color spaces
plt.figure(figsize=(4, 3))

plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original RGB Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale")
```

```
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original RGB Image

Grayscale



HSV

LAB



6. Perform basic geometric transformations such as resizing, rotation, and horizontal/vertical flipping.

```
# Resize the image to half of its original dimensions
new_width = width // 2
new_height = height // 2

resized_image = cv2.resize(
    image,
    (new_width, new_height)
)

print("Original size:", (width, height))
print("Resized size:", (new_width, new_height))

# Displaying Result
plt.figure(figsize=(3, 2))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original size: (217, 300)
Resized size: (108, 150)

Original Image

Resized Image



```
# Finding the center of the image
center = (width // 2, height // 2)

# Create a rotation matrix for 45 degrees
rotation_matrix = cv2.getRotationMatrix2D(
    center,
```

```

    45,
    1.0
)

# Applying the rotation
rotated_image = cv2.warpAffine(
    image,
    rotation_matrix,
    (width, height)
)

print("Image rotated by 45°")

# Displaying Result
plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image - 45°")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Image rotated by 45°

Original Image



Rotated Image - 45°



```

# Flip the image horizontally
horizontal_flip = cv2.flip(image, 1)

# Displaying Result
plt.figure(figsize=(3, 2))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Original Image Horizontal Flip



```

# Flip the image vertically
vertical_flip = cv2.flip(image, 0)

# Displaying Result
plt.figure(figsize=(3, 2))

```

```
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Vertical Flip



7. Generate the complement (negative) of the image by inverting pixel intensity values.

```
# Generating the complement (negative) image
# For an 8-bit image, negative = 255 - original pixel value
negative_image = 255 - image

plt.figure(figsize=(5, 4))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(negative_image, cv2.COLOR_BGR2RGB))
plt.title("Negative / Complement Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Negative / Complement Image



8. Crop a Region of Interest (ROI) from the image and analyze its characteristics.

```
# Defining the Region of Interest (ROI) coordinates
x1 = width // 4
y1 = height // 4

x2 = 3 * width // 4
y2 = 3 * height // 4

# Extracting ROI using NumPy slicing
roi = image[y1:y2, x1:x2]

print("ROI Shape:", roi.shape)
```

ROI Shape: (150, 108, 3)

```
# Display Region of Interest (ROI)
plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Region of Interest (ROI)



```
# Analyze the Region of Interest (ROI) properties or characteristics
roi_height, roi_width, roi_channels = roi.shape

print("ROI Properties:\n")
print("ROI Width:", roi_width)
print("ROI Height:", roi_height)
print("ROI Channels:", roi_channels)
print("ROI Shape:", roi.shape)
print("ROI Data Type:", roi.dtype)
```

ROI Properties:

```
ROI Width: 108
ROI Height: 150
ROI Channels: 3
ROI Shape: (150, 108, 3)
ROI Data Type: uint8
```

9. Display the original and processed images for comparison and observe the effects of each operation.

```
# Comparing the original and processed images
plt.figure(figsize=(5, 4))

plt.subplot(3, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(3, 3, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(3, 3, 3)
plt.imshow(cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(3, 3, 4)
plt.imshow(cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.subplot(3, 3, 5)
plt.imshow(cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB))
plt.title("Resized")
```

```
plt.axis("off")

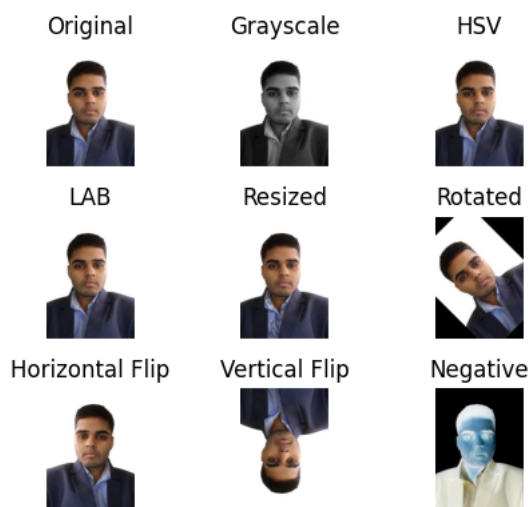
plt.subplot(3, 3, 6)
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(3, 3, 7)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(3, 3, 8)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.subplot(3, 3, 9)
plt.imshow(cv2.cvtColor(negative_image, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Saving the processed images
cv2.imwrite("grayscale_image.jpg", gray_image)
cv2.imwrite("resized_image.jpg", resized_image)
cv2.imwrite("rotated_image.jpg", rotated_image)
cv2.imwrite("horizontal_flip.jpg", horizontal_flip)
cv2.imwrite("vertical_flip.jpg", vertical_flip)
cv2.imwrite("negative_image.jpg", negative_image)
cv2.imwrite("roi_image.jpg", roi)

print("All processed images have been saved successfully.")
```

All processed images have been saved successfully.

10. Document the observations and discuss the significance of these preprocessing techniques in computer vision applications.

✓ **Answer:**

Observations

1. The original image contains three color channels, whereas the grayscale image contains only one channel.
2. OpenCV reads color images in BGR format, while Matplotlib uses RGB format for displaying images.
3. JPEG uses lossy compression, whereas PNG uses lossless compression.
4. Grayscale conversion reduces color information and simplifies further image processing.
5. HSV separates color information into Hue, Saturation, and Value, making it useful for color-based image processing.
6. LAB separates lightness from color information and is useful for color analysis and enhancement.
7. Resizing changes the dimensions of an image.
8. Rotation changes the orientation of the image.

9. Horizontal and vertical flipping produce mirror images.
10. The negative transformation reverses the intensity values of pixels.
11. ROI extraction allows a particular region of an image to be analyzed separately.

Significance in Computer Vision

Image preprocessing is an important initial stage of computer vision systems. It improves the quality and consistency of image data before applying advanced techniques such as feature extraction, object detection, image segmentation, classification, and deep learning.

These operations are commonly used in applications such as:

- Medical image analysis
- Autonomous vehicles
- Face recognition
- Surveillance systems
- Industrial defect detection
- Robotics
- Optical Character Recognition (OCR)

Q1. What is a digital image? Differentiate between grayscale and color images.

Answer:

A digital image is a two-dimensional representation of a visual scene consisting of a finite number of pixels. Each pixel contains numerical information representing the intensity or color at a particular location.

A grayscale image contains only one channel, where pixel values usually range from 0 to 255. A value of 0 represents black and 255 represents white.

A color image generally contains three channels representing color components. In RGB, these channels are Red, Green, and Blue. In OpenCV, color images are normally stored in BGR order.

Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

Answer:

RGB: Represents an image using Red, Green, and Blue channels. It is commonly used for displaying images.

BGR: Uses Blue, Green, and Red channels. OpenCV uses BGR as its default color ordering when reading images.

HSV: Represents color using Hue, Saturation, and Value. It is useful for color-based segmentation because hue represents the basic color independently from brightness.

LAB: Represents color using Lightness (L), and two color components, A and B. It is useful in color analysis and image enhancement because lightness is separated from color information.

Q3. What is the purpose of converting an image to grayscale before further processing?

Answer:

Grayscale conversion reduces a three-channel color image to a single intensity channel. This reduces computational complexity and memory requirements. Many image processing techniques such as edge detection, thresholding, contour detection, and feature extraction can be performed effectively using grayscale images.

Q4. Explain the concept of image complement (negative image) and mention its practical applications.

Answer:

An image complement or negative image is generated by reversing the intensity values of the pixels. For an 8-bit image, the transformation is:

$$s = 255 - r$$

where r is the original pixel intensity and s is the resulting intensity.

For example, a pixel with intensity 50 becomes 205.

Image negatives can be useful for enhancing details in certain images, medical image analysis, photographic applications, and visualization of structures that may be difficult to observe in the original image.

Q5. Differentiate between image resizing, cropping, and scaling.

Answer:

Resizing changes the dimensions of an image, such as changing an image from 1920×1080 to 960×540 .

Cropping removes unwanted portions of an image and extracts a selected region.

Scaling refers to changing the size of an image using a scaling factor. For example, scaling an image by 0.5 reduces both its width and height by half.

Q6. What is a Region of Interest (ROI), and why is it important in computer vision?

Answer:

A Region of Interest (ROI) is a specific portion of an image selected for further processing or analysis.

ROI is important because it allows algorithms to focus only on the relevant portion of an image. This reduces computational requirements and can improve the accuracy of tasks such as object detection, face recognition, medical image analysis, and industrial inspection.

Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

Answer:

OpenCV is widely used because it provides a large collection of optimized functions for image processing and computer vision. It supports image acquisition, filtering, transformations, feature detection, object detection, video processing, camera operations, and many other tasks.

It also works efficiently with NumPy arrays and provides interfaces for Python, C++, and other programming languages.

Q8. Explain how image resolution and pixel intensity influence image quality.

Answer:

Image resolution represents the number of pixels used to represent an image. Higher resolution generally provides more spatial detail, while lower resolution may result in loss of fine details.

Pixel intensity represents the brightness value of a pixel. In an 8-bit grayscale image, intensity values range from 0 to 255, where 0 represents black and 255 represents white.

Therefore, both spatial resolution and intensity information influence the visual quality and amount of information available in an image.

Q9. Mention five real-world applications where basic image preprocessing is an essential step.

Answer:

Five applications are:

- Medical imaging — preprocessing X-rays, CT scans, and MRI images.
- Autonomous vehicles — processing camera images before object detection.
- Face recognition — resizing, grayscale conversion, and normalization.
- Industrial inspection — detecting defects in manufactured products.
- Surveillance systems — preprocessing video frames before object or activity detection.

Other applications include satellite imaging, OCR, robotics, biometric systems, and agriculture.

Q10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Answer:

Image preprocessing improves the quality and consistency of input data before it is provided to computer vision algorithms. Operations such as resizing ensure that images have a common dimension, grayscale conversion reduces unnecessary information, normalization can provide consistent intensity ranges, and noise reduction can remove unwanted variations.

As a result, feature extraction algorithms can identify important patterns more effectively. Deep learning models can also train more efficiently when input images are consistent and appropriately prepared.